Riverfed.

A PRACTICAL TRAINING

The Riverfed Handbook

How to stop chatting with AI and start operating it.

53 sources | 21 concepts | 12 topics | 61 entities

IN ONE SENTENCE

You have been renting the AI. Today you install it.

Same model. Different place. That difference is the whole training.

Where the AI actually runs

The setup nobody explains, and the one demo that makes it click.

01

WHERE YOU ARE NOW

A tab, or a program on your machine

AI in a browser tab

Runs in someone else's cloud. You paste things in, you copy things out.

- Cannot see your files
- Cannot create anything you can open
- Cannot run anything
- You are the clipboard between it and your work

AI in your terminal

The same model, running next to your work, with your permission.

- Reads the files you point it at
- Writes new files onto your disk
- Runs commands and sees what they return
- You stop being the clipboard

The whole shift in one line: it stops being something you talk to and starts being something that does.

DEMO. DO THIS LIVE, DO NOT DESCRIBE IT

Two commands that change the room

01
Ask it to write

Type: `create a txt file saying hello Maria`. Use the name of someone actually sitting there.

02
The file appears

Show it in Finder or Explorer. A real file, on their kind of machine, with a classmate's name in it.

03
Ask it to open

Type: `now open the file`. It opens. That was not text: that was a command running on the computer.

Why a real name matters: it kills the "this is a canned demo" doubt before anyone says it out loud. Nobody prepared a file with Maria's name in it.

THE BRIDGE

A text file and a program are the same thing to it.

Writing .txt or writing .py is one skill. Opening a file or running a program is one skill. It has both.

A QUESTION THAT STOPS MATTERING

So which editor should you use?

VS CODE

Fine

Popular, free, huge extension ecosystem.

CURSOR

Also fine

An editor with AI built in. Convenient, not required.

NONE

Also fine

A terminal and any text editor is a complete setup.

The point: the agent runs beside your editor, not inside it. Which editor you open is a preference, not an architecture decision.

SO DO YOU ABANDON THE BROWSER?

No. You learn where the handoff is

Stay in the GUI

Lowest friction. Where thinking happens.

- Research and reading around a problem
- Ideation, drafting, arguing with yourself
- Simple automations
- Anyone who is not ready for a terminal yet

Escalate to the terminal

Full filesystem and shell. Where shipping happens.

- Work that spans many files
- Anything that has to run, not just read
- Deployment and real output
- Long jobs you want to leave running

The handoff point, not the tool choice, is the decision. They can share context, so moving between them is a continuation rather than a restart.

What it reads and writes

Plain text is the substrate. That is not a limitation, it is the point.

02

THE MOST EVIDENCED IDEA IN THE WHOLE WIKI

Markdown is the oxygen

Agents consume markdown: `agents.md`, `memory.md`, skills, `design.md`.
Agents produce markdown: reports, wiki pages, new skills. Knowledge persists as markdown.

- Human readable, so you can check its work
- Version controllable, so you can see what changed
- Parseable by every model, so nothing locks you in

The consequence: the quality of your markdown is the ceiling on your agent's intelligence. Better files, better output. Every session, every tool.

WHY A TERMINAL, SPECIFICALLY

The terminal is where things converge

Context, tools, and execution meet in one place. That is the whole argument. It is not about looking technical.

- Your context files are right there on disk
- Tools compose: the output of one becomes the input of the next
- Execution is local, so results are real rather than described
- It is the lowest common denominator across every agent harness

The practical upshot: invest in context-file conventions, not in learning a specific GUI. The conventions move with you. The GUI does not.

THE COMPLAINT YOU WILL HEAR MOST

"It keeps forgetting everything"

What people assume

The model has a bad memory. Nothing to be done about it.

- Blame the model
- Repeat yourself every session
- Conclude agents are not ready

What is actually happening

Memory was never written down. Retrieval is not the broken part.

- Check `memory.md` exists
- Check something is writing to it
- Check compaction is not discarding it first

Memory failures are write failures. If your agent forgets a correction, the bug is in what got written, not in what got recalled.

WHERE TO PUT YOUR EFFORT

Manage the vault, not the agent

Invest in organising the knowledge, not in configuring the tool. Daily logs, project notes, style guides. A structure built for your own thinking that the agent happens to read.

- Harnesses change. Agents forget. Context windows reset.
- A folder of markdown outlives all three.
- No lock-in: any agent, any model, can read it.
- If you organise it well enough to understand yourself, the agent inherits that.

The test: if you switched agent tools tomorrow, how much of your setup would survive? Vault work survives. Configuration does not.

Getting good at it

The loop that separates a demo from a system you trust.

03

THE WIKI'S THIRD MOST EVIDENCED IDEA

Do it, formalize it, iterate it

01

Do it manually

Walk the real workflow with the agent. Correct it as you go. Keep going until the output is actually right.

02

Formalize the run

Have the agent package that successful run into a reusable skill file. It captures the corrections you made.

03

Iterate on use

Each new use exposes a gap. Fold it back in. Quality compounds toward a version 5 or 10 that genuinely works.

Why not just write the skill up front? A hand-written or downloaded skill is a context-free SOP. It never met the edge cases. A skill born from a real run inherits every correction you made during it.

THE QUESTION EVERYONE ASKS AT THIS POINT

Autonomy is earned, not configured

Not a switch you flip. A ladder the agent climbs one bounded workflow at a time. Start at the bottom rung: fixed workflow, it drafts, you approve.

- Bottom rung is shippable on day one, because it is just a workflow
- Promotion criterion: demonstrated quality at the current rung
- Grant more judgment only after it has earned it, the way you would with a new hire
- "It drafts, you approve, until it stops needing you" is a story people trust

Why this matters commercially: full autonomy is the value, but full autonomy on day one is how you get slop, broken trust, and cancelled pilots.

THE PART ALMOST EVERYONE SKIPS

If you cannot measure it, you cannot promote it

Evals are a set of real examples with known right answers, scored by a rubric or by another model. They turn "is it any good?" from vibes into a number.

- Track that number across every prompt, model, or tool change
- It is the promotion criterion for the autonomy ladder
- It catches quality drift before a customer does
- The eval set is both your QA loop and your sales evidence

The dependency: autonomy is only sellable when quality is visible. Evals are what convert "trust me" into evidence.

How it is put together

Same four pieces underneath, whichever product you picked.

04

THE ANATOMY

Four pieces of every production agent

01

A model

The reasoning. Swappable, and cheaper every quarter.

02

Tools

How it touches the world: files, shell, APIs, a browser.

03

A loop

Act, observe the result, decide again. Not one shot.

04

Context

What it knows going in: your files, your instructions, your memory.

Where your leverage is: you do not control the model and you barely control the loop. You control the tools you expose and the context you write. That is the job.

AN ARCHITECTURE DECISION WITH A PRICE TAG

CLI first, API second, MCP last

~200

tokens a CLI returns for a query: scoped, pre-formatted, out of your way

~132K

tokens an MCP can dump for the same query, as raw JSON

every

turn an MCP re-sends its tool schemas,
whether you use them or not

Two costs, not one. Tokens are an operating expense, and reliability drops as well: the more an agent has to parse, the more it loses the thread.

SAME ENGINE, DIFFERENT CAR

The harnesses people actually use

37
Claude Code
Terminal-native. The reference for this deck.

19
Codex
GUI plus CLI. Strong on long runs.

18
OpenClaw
Open harness, bring your own model.

8
Cowork
GUI-first. Where the handoff often starts.

6
Hermes
Always-on personal assistant framing.

3
Cursor
The editor with the agent built in.

Read the numbers, not the names. That is how many independent sources in this wiki discuss each one. The list will look different in six months, which is exactly why the four pieces matter more.

HOW OTHER PEOPLE GET TO USE IT

A dashboard is the access layer, not the system

Terminal-first agents create leverage for one power user and exclude everyone else. A dashboard turns hidden agent work into something a teammate can see and run.

- Status, queues, and metrics for runs nobody watched
- Buttons and approvals for people who should never touch a prompt
- Handoff points, so a stalled run gets noticed
- A packaging surface: it is how this becomes something you can sell

The failure mode has a name: dashboard theater. A polished control panel with no reliable workflow underneath it. Build the workflow first.

WHEN ONE AGENT IS NOT ENOUGH

Long jobs fail in three specific ways

On a long task in a single context window, agents degrade predictably. Naming the three failure modes makes them diagnosable instead of mysterious.

- **Stops early.** Declares the job done when it is not.
- **Favours its own work.** Ask it to judge its output and it approves.
- **Loses the objective.** Context grows, compacts, and the original goal fades. Context rot.

The fix: have the agent write its own team for the job. Sub-agents with clean contexts, one job each, judged separately. It costs real tokens, which is the trade you are making.

What it costs, and against what

05

The objection you will hear in every budget meeting.

THE REFRAME

Price it against the hours, not the tokens

$150

of tokens for a board memo that would have taken hours of analyst time

$40K

a month in model spend, replacing several full-time salaries

0×

marginal cost of the second, fifth, or fiftieth variant

Token cost is the wrong denominator. The right one is the human time replaced, plus the overhead and onboarding you never paid.

THE DISCIPLINE NOBODY WANTS

Ask first, build second

Building got cheap, so the temptation to build is higher than it has ever been. The cost of building the wrong thing did not fall at all.

- Check whether a tool, plugin, or platform already does it
- Custom builds carry maintenance and context rot that off-the-shelf does not
- Three days on a proposal generator is three days not spent on customers
- The cheaper building gets, the more valuable this discipline becomes

The trap: being able to build anything in an afternoon feels like leverage. It is only leverage if the thing needed to exist.

TURNING A TASK INTO A SYSTEM

Loop engineering

01

One objective metric

Black or white. A ranking, a score, revenue, a rubric result. Without one the agent loops forever.

02

A stop condition

What good enough looks like. This is what makes long-running autonomy safe to leave scheduled.

03

Memory between runs

Each run writes what it learned to a markdown file. The next run starts from there instead of from zero.

The difference this makes: an agent that does a task once versus a system that compounds. Run monthly for years, and each expensive run becomes an asset for the next one.

Taste is the part that does not commoditize

Everyone has the same models. The inputs are what differ.

06

THE MOST LEVERAGED FILE YOU OWN

The templates renaissance

Code and content are commoditized. A well-structured template is not: it hands the agent structure, voice, and domain conventions from the very first token.

- Without one, an agent starts from zero and produces generic output
- With one, it inherits your standards before it writes a word
- Starter files, boilerplates, storyboard formats, style guides
- What used to be dismissed as boilerplate is now the highest-leverage thing you maintain

The rule: the quality ceiling of agent-produced work is proportional to the quality of the template it started from. Fix the template, not the output.

A TRICK WORTH STEALING

Show it, do not describe it

Text-only brief

You describe what you want in words and hope it lands.

- Palette and spacing get lost
- "Clean and modern" means nothing
- You iterate by arguing about adjectives

Visual-first brief

Screenshots and reference images become the spec.

- A multimodal model reads the images
- It writes the structured brief
- An agent executes it

Why it works: images carry design intent, palette, layout, typography, spacing, that text-only prompts consistently fail to convey.

What this does to the work

07

Roles, distribution, and how the first sale actually happens.

TWO FORCES AT ONCE

Roles are collapsing in two directions

Horizontal collapse

The coder, the PM, and the designer merge into one builder.

- Each can now do the other two's work
- One person with judgment spans functions
- Specialisation stops being the hiring signal

Vertical shift

The individual contributor becomes a manager of agents.

- You direct a fleet rather than doing the work
- The skill is delegation and review
- Your throughput stops being bounded by your hands

What hiring becomes: not "can this person code, design, or manage a product?" but "can this person direct agents and judge the result?"

THE UNCOMFORTABLE ONE

Distribution beats engineering now

Code is commoditized: hundreds of thousands of new vibe-coded projects a day. The durable advantage moved from building to getting it in front of people.

- The correct sequence inverts: audience first, product second
- Founders who spend half their time on traffic and selling outperform those optimising code
- An hour building without a distribution plan is procrastination with a nice IDE
- The wealthiest builders in this era will be marketers

Sharp version: a product nobody knows about is a hobby, and agents just made building the cheap part of the equation.

HOW THE FIRST SALE ACTUALLY HAPPENS

Lead with a cheap diagnostic

01

Sell a small audit

Cheap or free. Low risk, high clarity. The buyer is deciding on a small thing, not a big one.

02

Prove value inside it

Find something measurable. You also learn the real, undocumented workflow before you ever quote a build.

03

Let the proof pull them up

The client asks for the implementation. Then the retainer. You never had to make the hard first sale.

Why it works: when intelligence and code are commoditized, the scarce thing is a trusted path into a specific business. An audit buys that.

Where this stops working

08

One source in this wiki reports a failure. It is the most useful one.

THE LIGHTS-OFF EXPERIMENT

Nobody read the code, and it fell apart

An agent software factory run with nobody reading the output. It worked, until an issue appeared that no amount of prompting could fix, the site was down, and the fix meant digging through a codebase abandoned months earlier.

- Not a skill problem, and not a scale problem
- No harness and no extra tokens fixes it
- Coding models are rewarded for one thing: did the test pass without breaking another
- Nothing in that reward penalises bad architecture, whose cost arrives months later

The uncomfortable part: they get better at passing tests and no better at keeping a codebase maintainable. If a model knew what good looked like, it would already write it.

SO WHAT DO YOU DO ABOUT IT

Turn the lights back on

The fix is not to use agents less. It is to keep a human in the parts the reward function cannot see.

- Plan up front: product review, then system architecture
- Program design down to types and call graphs, the step everyone skips
- Then vertical slices, so each piece is reviewable
- Keep reading the output. You can still move fast while doing it.

The trade: thirty minutes of alignment up front saves hours of review afterwards, and a good result becomes a joy to read instead of slop to untangle.

THE WHOLE TRAINING IN ONE LINE

Write the context. Judge the output.
Everything else is the tool's job.

The models improve without you. Your files and your judgment are the parts that compound.

IF YOU DO NOTHING ELSE

Minimum viable operator

01

Install it locally

One agent, running in a terminal, in a folder you care about.

02

Start one file

A markdown file it reads every session. Add to it when you correct it.

03

Formalize one workflow

Do a real job manually, then have it package the run into a skill.

04

Keep twenty examples

Real cases with the answer you would have given. That is your eval set.

Note what is missing: no MCP servers, no dashboard, no fleet. Those come after this works, not before.

LAST THING

Go and give it something real to do.

Not a test. Something from your actual work, today, that you would otherwise have done by hand.